ENGINEERING OWNERSHIP MANUAL

AegisRAG
AI Incident Commander

Architecture, implementation, technology decisions, trade-offs, evaluation, safety, scaling, and interview defense.

RAG	LANGGRAPH	MULTI-TENANCY	INCIDENT INTELLIGENCE

Built to help the project author explain every important design choice from first principles - without overstating production usage.

Interview edition | Generated 2026-08-26

Contents

Use this as a study sequence: understand the system, trace one incident, defend the retrieval and orchestration choices, then rehearse the interview questions.

Purpose of this guide 6

1. The shortest accurate explanation 6

2. The problem being solved 6

3. System context 6

- Local deterministic profile . . . 7
- Production-style profile . . . 7

4. Technology choices 7

- FastAPI and Pydantic . . . 7
- LangGraph . . . 7
- LangChain Core . . . 8
- Qdrant . . . 8
- Redis . . . 8
- OpenTelemetry and Jaeger . . . 8
- Technology decision record: choices and rejected alternatives . . . 8
  - Python 3.11 instead of Java, Go, or TypeScript . . . 8
  - FastAPI instead of Flask or Django . . . 8
  - Pydantic instead of plain dictionaries or dataclasses . . . 8
  - LangGraph instead of a linear LangChain chain . . . 8
  - LangGraph instead of CrewAI or AutoGen . . . 9
  - LangChain Core instead of framework-free provider calls . . . 9
  - Qdrant instead of FAISS . . . 9
  - Qdrant instead of Pinecone, Weaviate, Elasticsearch, or pgvector . . . 9
  - Custom local retrieval instead of pretending the offline path is production ML . . . 9
  - Redis instead of in-process caching . . . 9
  - SQLite locally and PostgreSQL in production . . . 9
  - OpenTelemetry and Jaeger instead of vendor-specific tracing . . . 10
  - YAML scenarios instead of random event generation . . . 10
  - Pytest and Ruff instead of a larger toolchain . . . 10
  - Docker Compose instead of Kubernetes in the repository . . . 10
  - Optional OpenAI generation instead of a mandatory hosted model . . . 10
- How to answer any stack-choice question . . . 10

5. Repository map 10

6. End-to-end payments incident walkthrough 11

Step 1: Load and validate the scenario . . . 11
Step 2: Seed operational knowledge . . . 11
Step 3: Generate causal events . . . 11
Step 4: Evaluate the alert . . . 11
Step 5: Build the investigation request . . . 12
Step 6: Run the LangGraph workflow . . . 12
Step 7: Rank hypotheses . . . 12
Step 8: Persist and verify . . . 12

7. Ingestion in detail **12**

Content identity . . . 12
Structure-aware chunking . . . 12
Why provenance matters . . . 12
Production improvement . . . 12

8. Hybrid retrieval from first principles **12**

Lexical retrieval . . . 12
Dense retrieval . . . 13
Reciprocal Rank Fusion . . . 13
Maximal Marginal Relevance . . . 13
Production Qdrant path . . . 13

9. LangGraph investigation **13**

Graph state . . . 13
Retrieve node . . . 13
Grade evidence node . . . 13
Rewrite node . . . 13
Generate node . . . 13
Verify node . . . 13
Why one retry . . . 14

10. Incident detection and classification **14**

11. Multi-tenancy **14**

12. Persistence and checkpointing **14**

13. Caching **14**

14. Observability **15**

15. Evaluation **15**

Recall at K . . . 15
Mean Reciprocal Rank . . . 15
nDCG at K . . . 15

Current verified results . . . 15

16. Test strategy **15**

17. Safety and threat model **16**

Prompt injection . . . 16
Unsafe remediation . . . 16
Cross-tenant leakage . . . 16
Sensitive telemetry . . . 16

18. Important trade-offs **16**

Why custom retrieval plus frameworks? . . . 16
Why SQLite locally? . . . 16
Why deterministic simulation? . . . 16
Why not fully autonomous remediation? . . . 16
Why not Kafka in the default path? . . . 17

19. Known limitations **17**

20. Production scaling plan **17**

Phase 1: identity and data durability . . . 17
Phase 2: event ingestion . . . 17
Phase 3: retrieval quality . . . 17
Phase 4: safe operations . . . 17

21. How to demo the project **18**

Five-minute demo sequence . . . 18
Two-minute verbal pitch . . . 18

22. Interview questions and strong answers **18**

What did you personally build? . . . 18
Why is this RAG rather than ordinary search? . . . 18
Why use both lexical and dense retrieval? . . . 18
Why RRF instead of adding BM25 and cosine scores? . . . 18
Why apply MMR after RRF? . . . 18
Why LangGraph? . . . 19
Where is LangChain used? . . . 19
How do you prevent hallucinated citations? . . . 19
How is tenant isolation enforced? . . . 19
Why is x-tenant-id not enough for production? . . . 19
How are incidents generated? . . . 19
Why require consecutive violating samples? . . . 19
How does the healthy control help? . . . 19
How is idempotency handled? . . . 19

What happens when evidence is weak? . . . 19
How would you make confidence statistically meaningful? . . . 19
Why does readiness check dependencies but liveness does not? . . . 20
What happens if Qdrant succeeds but metadata persistence fails? . . . 20
How would you support millions of events per minute? . . . 20
How would you invalidate Redis cache entries? . . . 20
Why not store all telemetry in Qdrant? . . . 20
How would you evaluate generation? . . . 20
What is the biggest current weakness? . . . 20
What would you build next? . . . 20

23. Code-reading path for ownership **20**

24. Commands to memorize **21**

25. Honest resume claims **21**

Final ownership checklist **21**

Purpose of this guide

This guide is designed to make you the genuine owner of AegisRAG. By the end, you should be able to:

- Explain the product in thirty seconds, two minutes, and fifteen minutes.
- Trace an incident from generated telemetry to a persisted investigation report.
- Explain BM25-style lexical retrieval, dense retrieval, Reciprocal Rank Fusion, and Maximal Marginal Relevance.
- Defend the use of LangGraph and LangChain without treating either framework as magic.
- Explain multi-tenancy, checkpointing, caching, observability, safety, evaluation, and deployment.
- Discuss honest limitations and a credible production scaling plan.
- Answer common SDE II, RAG, distributed-systems, and system-design interview questions.

1. The shortest accurate explanation

AegisRAG is an independently built, production-style incident intelligence platform. It simulates tenant-isolated production systems, generates causal deployment and telemetry events, evaluates alert rules, creates incidents, and runs a checkpointed LangGraph investigation. The investigation retrieves runbooks, postmortems, and service ownership data through hybrid retrieval; ranks root-cause hypotheses; verifies every citation; and stops before privileged remediation unless a human approves it.

The important qualifier is production-style. The repository runs end to end and exercises real architecture boundaries, but it does not claim to process telemetry from a real employer or real customers.

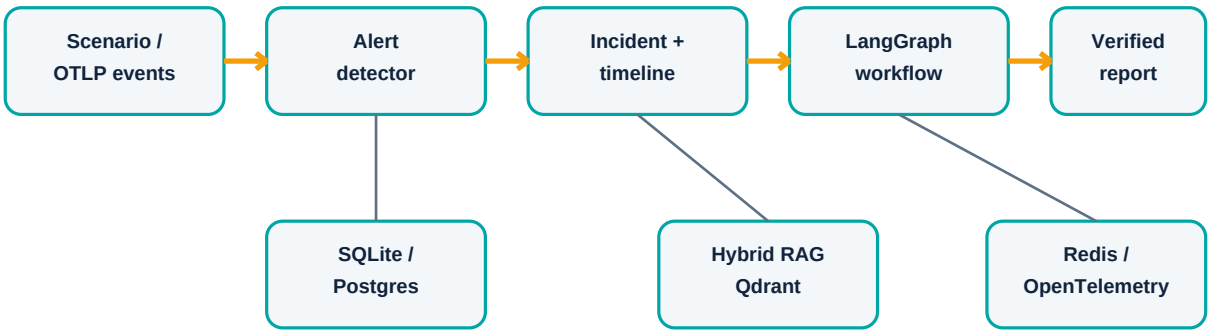
2. The problem being solved

During an outage, engineers must combine several weak and fragmented signals:

- What changed recently?
- Which service is failing?
- Which dependencies are saturated?
- Has this failure happened before?
- Which runbook applies?
- Is the proposed mitigation safe?
- What evidence supports the diagnosis?

A normal document chatbot only answers questions over static text. AegisRAG connects a causal incident lifecycle to retrieval and reasoning. Telemetry crosses a rule threshold, an incident is created, relevant operational knowledge is retrieved, and the result becomes a stored, auditable timeline.

3. System context



Every request and persistence boundary carries tenant identity and provenance.

Figure 1. Executable local path and replaceable production adapters.

The system has two operating profiles.

Local deterministic profile

- SQLite stores documents, chunks, telemetry, incidents, audit events, and timelines.
- A deterministic hashing embedder avoids external model calls.
- A custom hybrid retriever combines lexical and dense rankings.
- LangGraph uses an in-memory checkpointer unless a checkpoint path is configured.
- A deterministic extractive answer provider generates cited answers.
- No API key or external service is required.

This profile exists so CI and interview demos are reproducible.

Production-style profile

- Qdrant stores named dense and sparse vectors and performs tenant-filtered RRF.
- Redis caches tenant-scoped query responses.
- OpenTelemetry exports traces to Jaeger.
- LangChain/OpenAI can provide structured answer generation.
- Docker Compose starts the API, Qdrant, Redis, and Jaeger.
- PostgreSQL checkpoint support is included as a production dependency and architecture path.

4. Technology choices

FastAPI and Pydantic

FastAPI provides typed request handling, generated OpenAPI documentation, dependency-friendly routing, and clear health/readiness endpoints. Pydantic validates tenant IDs, document sizes, questions, scenario definitions, telemetry events, incident reports, and workflow metadata.

LangGraph

LangGraph models the investigation as explicit state transitions rather than an opaque agent loop. It provides conditional routing and checkpointable thread state. The graph is bounded: evidence can trigger one query rewrite, then the workflow must generate or abstain.

LangChain Core

LangChain is used at the interface boundary. The custom retrieval engine implements LangChain's BaseRetriever contract and returns standard Document objects. This gives interoperability without hiding the retrieval algorithm behind a framework default.

Qdrant

Qdrant is the production vector backend because it supports named dense and sparse vectors, payload filtering, and server-side Reciprocal Rank Fusion. Tenant filtering is applied to both retrieval legs before fusion.

Redis

Redis caches query responses using a hash of tenant ID, question, and top K. Tenant identity is part of the key, preventing one tenant from receiving another tenant's cached answer.

OpenTelemetry and Jaeger

OpenTelemetry is the vendor-neutral instrumentation layer. Jaeger is the local trace backend. Together they make API latency and dependency paths inspectable without coupling the application to one monitoring vendor.

Technology decision record: choices and rejected alternatives

The strongest interview answer is not "this tool is popular." It is: "these were my requirements, this tool satisfied them, this was the cost, and this is the point at which I would replace it."

Python 3.11 instead of Java, Go, or TypeScript

Python has the deepest current ecosystem for retrieval, embedding providers, evaluation, and agent orchestration. It also keeps the local demo compact. Java or Go would provide stronger compile-time guarantees and often better CPU efficiency, while TypeScript would be attractive for a full-stack team. None of those advantages dominates here because the latency-heavy operations are storage and model calls. Python 3.11 gives modern typing and good runtime performance while remaining compatible with the selected AI libraries. In a large production organization, a Go or Java telemetry ingestion service could feed a Python investigation service; the architecture does not require one language for every workload.

FastAPI instead of Flask or Django

FastAPI was selected because the API is contract-heavy and asynchronous integration is likely. It turns Pydantic models into request validation and OpenAPI schemas, supports dependency injection, and works naturally with ASGI. Flask is smaller but would require assembling validation, schema generation, and async conventions. Django is excellent for a database-centered product with an ORM and admin console, but is heavier than this service needs. The trade-off is that FastAPI does not provide Django's mature ORM and admin features, so production persistence should add an explicit database layer and migrations.

Pydantic instead of plain dictionaries or dataclasses

Events, scenarios, requests, and reports cross trust boundaries. Pydantic rejects malformed types and enforces bounds before data reaches business logic. Standard dataclasses are useful for internal domain objects but do not provide equivalent parsing, JSON Schema generation, and environment-settings integration without additional libraries. The cost is validation overhead and framework coupling at boundaries, so core ranking functions still use simple typed Python structures where possible.

LangGraph instead of a linear LangChain chain

The workflow needs conditional routing, bounded retry, checkpointed state, auditable node traces, and a future human-approval pause. A linear chain can perform retrieve-then-generate, but branching and resumption quickly become manual control-flow code. LangGraph makes the state machine explicit. The cost is additional state serialization and graph concepts. If the workflow were only one retrieval call and one model call, LangGraph would be unnecessary.

LangGraph instead of CrewAI or AutoGen

CrewAI and AutoGen emphasize collaborating role-based agents and conversational autonomy. This incident workflow needs predictability more than agent personas: one bounded graph, explicit state, deterministic verification, and no open-ended delegation. LangGraph is therefore easier to test and reason about. A multi-agent framework could become useful if separate security, database, and network investigators needed independent tools, but it would also increase latency, cost, and failure combinations.

LangChain Core instead of framework-free provider calls

LangChain Core supplies standard `Document` and `BaseRetriever` contracts plus structured-output integration. That makes the custom retriever usable by ecosystem components while the important ranking logic remains visible. Direct SDK calls would reduce dependencies but create custom adapters around every provider. Conversely, using a high-level prebuilt RAG chain would hide the ranking and verification behavior that this project is meant to demonstrate. The project uses the smallest useful LangChain surface rather than making it the architecture.

Qdrant instead of FAISS

FAISS is a high-performance nearest-neighbor library, not a complete multi-tenant vector service. It does not by itself provide a network API, durable collection management, rich payload filtering, or hybrid sparse-dense query execution. Qdrant provides those operational capabilities and server-side RRF. FAISS would be appropriate for a single-process prototype or a specialized in-memory index.

Qdrant instead of Pinecone, Weaviate, Elasticsearch, or pgvector

Qdrant is open source, runs locally in Docker, supports payload filters and named dense/sparse vectors, and keeps the demo reproducible without a managed account. Pinecone reduces operations for a managed deployment but adds vendor dependency and cost. Weaviate is a strong comparable choice with hybrid retrieval and a broader object model; Qdrant's API and local footprint fit this repository more directly. Elasticsearch/OpenSearch is compelling when lexical search, log analytics, and an existing operations footprint dominate; it is heavier for this self-contained demo. PostgreSQL with pgvector simplifies transactional metadata plus vectors and may be the best early production choice at moderate scale, but dedicated Qdrant keeps vector retrieval capabilities explicit. The correct production decision depends on corpus size, filtering, latency, operations expertise, and existing infrastructure.

Custom local retrieval instead of pretending the offline path is production ML

The local engine implements BM25-style lexical scoring, deterministic hashing vectors, RRF, and MMR. Its purpose is repeatability, algorithm visibility, and zero external credentials. It is not presented as equivalent to a modern embedding model. This two-profile design lets every reviewer run the project while preserving a credible production integration. The replacement boundary is the retriever interface, so changing the embedding or vector provider does not rewrite the graph.

Redis instead of in-process caching

An in-process dictionary would be simplest but would produce inconsistent caches across API replicas and disappear on restart. Redis provides shared TTL state, predictable operations, and familiar production behavior. The local profile intentionally uses no cache so tests do not depend on a daemon. Redis is not the system of record; cache loss may hurt latency but must not affect correctness.

SQLite locally and PostgreSQL in production

SQLite makes cloning, testing, and demoing the complete path nearly frictionless. WAL mode improves local concurrency, but SQLite remains a single-file database with limited multi-writer scaling. PostgreSQL is the production target for metadata, audit records, incidents, and durable graph checkpoints because it provides concurrent transactions, replication options, migrations, indexing, and row-level security. Keeping SQL persistence behind a store boundary makes that migration tractable.

OpenTelemetry and Jaeger instead of vendor-specific tracing

OpenTelemetry provides portable instrumentation and context propagation; Jaeger gives a free local UI and OTLP ingestion. Datadog, New Relic, Honeycomb, or Grafana Tempo could receive the same telemetry in production. Starting with a vendor SDK would be convenient but would couple application instrumentation to a purchasing decision. Metrics and logs still need a production backend; Jaeger is not a complete observability platform.

YAML scenarios instead of random event generation

YAML is readable in code review and easy to validate into typed definitions. Deterministic scenarios make cause, timing, expected alert behavior, and expected evidence reproducible. Random load generation is valuable for fuzzing and capacity work, but it is a poor primary regression oracle. Production ingestion would consume OTLP and deployment events; scenario YAML is a simulator and test fixture, not the intended enterprise transport.

Pytest and Ruff instead of a larger toolchain

Pytest offers concise fixtures, parametrization, and ecosystem support. Ruff provides fast linting and import checks in one binary. Mypy or Pyright, security scanners, mutation testing, and coverage gates would strengthen a production CI pipeline, but the current baseline prioritizes a fast signal that contributors will actually run. The Makefile gives reviewers memorable commands while CI invokes the same underlying tools.

Docker Compose instead of Kubernetes in the repository

Compose demonstrates service topology, dependency configuration, health checks, volumes, and ports on one machine. Kubernetes manifests or Helm charts would add deployment depth but cannot prove production readiness without a real cluster, secret management, autoscaling data, and operations ownership. A production evolution would package the same API image into Kubernetes, use managed PostgreSQL/Redis/vector services, and add probes, resource requests, disruption budgets, network policies, and GitOps deployment.

Optional OpenAI generation instead of a mandatory hosted model

The deterministic extractive provider makes tests repeatable, private, and free. The optional structured OpenAI path demonstrates realistic model integration without making repository correctness depend on network availability, API keys, model drift, or rate limits. A production model decision should consider quality, data residency, latency, cost, context limits, and provider fallback. The graph and verifier remain provider-independent.

How to answer any stack-choice question

Use this five-part pattern:

1. State the requirement the component owns.
2. Explain the chosen tool's concrete capability.
3. Name one credible alternative and when it would win.
4. Admit the chosen tool's operational or design cost.
5. Identify the interface that contains replacement risk.

For example: "I chose Qdrant because I needed tenant-filtered dense and sparse retrieval with server-side rank fusion and a locally runnable service. pgvector could win if the corpus were moderate and transactional simplicity mattered more. A dedicated vector service adds another consistency and operations boundary, so ingestion IDs and reconciliation must be explicit. The retriever protocol contains that choice."

5. Repository map

- `src/aegis_rag/api.py`: application assembly, middleware, routes, readiness, and dependency selection.
- `src/aegis_rag/scenarios.py`: YAML loading, deterministic event generation, alert evaluation, incident creation, and expected-outcome verification.
- `src/aegis_rag/workflow.py`: LangGraph state and retrieve-grade-rewrite-generate-verify control flow.

- `src/aegis_rag/incidents.py`: severity classification, hypothesis prioritization, confidence, actions, and approval boundary.
- `src/aegis_rag/retrieval.py`: local BM25-style ranking, dense similarity, RRF, MMR, and prompt-injection filtering.
- `src/aegis_rag/qdrant_store.py`: production dense-sparse indexing, tenant filtering, and native RRF.
- `src/aegis_rag/ingestion.py`: structure-aware section parsing, chunking, hashing, embeddings, and provenance.
- `src/aegis_rag/store.py`: SQLite schema and tenant-safe document, telemetry, incident, timeline, and audit persistence.
- `src/aegis_rag/generation.py`: deterministic extractive and optional structured OpenAI providers.
- `src/aegis_rag/evaluation.py`: Recall at K, MRR, nDCG, and quality thresholds.
- `src/aegis_rag/cache.py`: no-op local cache and Redis adapter.
- `src/aegis_rag/checkpointing.py`: in-memory or SQLite LangGraph checkpoint selection.
- `src/aegis_rag/telemetry.py`: optional OTLP trace configuration.
- `data/scenarios`: causal incident and healthy-control definitions.
- `data/knowledge`: runbooks, postmortems, service catalog, and operating policies.
- `data/eval/golden.jsonl`: labeled retrieval questions.
- `tests`: API, retrieval, evaluation, workflow, checkpoint, Qdrant, and scenario tests.

6. End-to-end payments incident walkthrough

The `payments-pool-exhaustion` scenario belongs to tenant `acme-payments`.

Step 1: Load and validate the scenario

`ScenarioRunner` loads YAML with `yaml.safe_load` and validates it using `ScenarioDefinition`. The definition contains tenant, service, environment, knowledge sources, ordered events, an alert rule, and expected evidence.

Step 2: Seed operational knowledge

The runner ingests the payments runbook and service catalog for the scenario tenant. Content is sectioned, chunked, embedded, content-hashed, and stored. If Qdrant is configured, the same chunks are indexed there.

Step 3: Generate causal events

The scenario creates:

1. Deployment `payments-api:v42`.
2. Database pool utilization at 0.85.
3. Pool wait p99 at 1800 milliseconds.
4. A `db_pool_timeout` log.
5. HTTP 503 rate at 0.24.
6. HTTP 503 rate at 0.42.

Every event has tenant, service, environment, run ID, scenario ID, event type, timestamp, name, value, and attributes. Events are persisted before alert evaluation.

Step 4: Evaluate the alert

The alert requires `http.server.503.rate > 0.10` for two samples. One violating sample is insufficient. The second violating sample opens the incident. This models basic false-positive resistance.

Step 5: Build the investigation request

The runner converts recent events into a normalized incident question containing service, environment, symptoms, and telemetry. It creates an `INC-SIM-*` identifier and invokes `IncidentCommander`.

Step 6: Run the LangGraph workflow

The graph retrieves evidence, grades it, optionally rewrites the query once, generates a cited answer, and verifies citation IDs. For the payments scenario, evidence is sufficient on the first attempt.

Step 7: Rank hypotheses

Evidence sections containing terms such as root cause, exhaustion, or failure receive higher hypothesis priority. Runbooks and postmortems receive a positive domain boost. Catalog and safety sections are useful context but are penalized as root-cause candidates.

Step 8: Persist and verify

The final report and timeline are stored under the tenant. Expected terms such as `connections are exhausted` and `stop the rollout` must appear in cited evidence. The CLI exits non-zero when expected evidence is missing.

7. Ingestion in detail

Content identity

Every document gets a SHA-256 content hash. Its deterministic ID includes tenant, source URI, and content hash. Re-ingesting identical content is recognized as a duplicate.

Structure-aware chunking

Markdown headings define sections. Paragraphs are accumulated until the configured character budget is reached. Chunks retain section name, ordinal, source document, start/end offsets, metadata, content hash, and embedding.

Why provenance matters

An answer citation must identify the exact source content used during retrieval. Content hashes allow an operator to determine whether a cited chunk came from the current or an older source version.

Production improvement

For large files, ingestion should move to asynchronous workers with object storage for originals, a durable queue, retry policies, and an outbox for metadata/index consistency.

8. Hybrid retrieval from first principles

Lexical retrieval

Lexical search rewards exact token matches. It is essential for error codes, service names, incident IDs, deployment SHAs, and configuration keys.

The local engine uses a BM25-style score. Important ideas are:

- Term frequency: repeated terms are more relevant, with saturation.
- Inverse document frequency: rare terms carry more weight.
- Length normalization: long chunks do not win merely because they contain more words.
- Compound normalization: `payments-api` also contributes `payments` and `api`.
- Small aliases normalize terms such as caused to cause.

Dense retrieval

Dense retrieval compares a query vector with chunk vectors using cosine similarity. It finds semantically related wording that may not share exact tokens.

The local hashing embedder is deterministic and dependency-free. It is an engineering fallback, not a claim that feature hashing matches a modern embedding model.

Reciprocal Rank Fusion

RRF combines rankings instead of raw scores:

```
RRF(document) = sum(1 / (k + rank_i(document)))
```

This is appropriate because BM25 and cosine scores have incompatible scales. A document ranking highly in both lists receives a stronger fused score.

Maximal Marginal Relevance

MMR balances relevance and diversity:

```
MMR = lambda * normalized_relevance - (1 - lambda) * max_similarity_to_selected
```

Without MMR, several near-duplicate chunks can consume the context window. The implementation normalizes relevance first so the diversity penalty does not numerically dominate small RRF scores.

Production Qdrant path

Each Qdrant point stores a dense vector, sparse vector, tenant payload, and chunk provenance. The query sends dense and sparse prefetches with the same tenant filter, then Qdrant performs RRF.

9. LangGraph investigation

Graph state

State contains tenant ID, original/current question, top K, retrieved hits, latency, candidate count, attempt count, evidence sufficiency, generated answer, grounded/abstained flags, citation-verification result, and node trace.

Retrieve node

The node creates a LangChain-compatible retriever and invokes it with the current question. Search hits are serialized into graph state so checkpoints remain inspectable.

Grade evidence node

Evidence is sufficient only when hits exist and the leading fused score exceeds the configured floor. A production version would use calibrated thresholds or a learned relevance grader.

Rewrite node

Weak evidence receives one operational query expansion containing root-cause, runbook, remediation, error-code, deployment, and dependency vocabulary.

Generate node

The local provider extracts overlapping evidence sentences. The optional OpenAI provider requests structured output containing answer text, cited evidence IDs, and grounded status.

Verify node

The verifier parses every `[C<number>]` marker and confirms that each number maps to selected evidence. Generation cannot declare itself grounded. Invalid citation references convert the response to an abstention.

Why one retry

During an incident, bounded latency and predictable cost matter. Unlimited reflection loops are difficult to debug and can drift away from evidence.

10. Incident detection and classification

Scenario alerts use an explicit metric, comparator, threshold, and required occurrence count. The code maps comparator strings to Python operator functions; it does not evaluate arbitrary expressions.

Severity is currently deterministic:

- SEV-1: outage, unavailability, all requests failing, or data loss.
- SEV-2: latency, timeout, degradation, or elevated error rate.
- SEV-3: lower-impact conditions.

This is intentionally understandable. A production system would combine impact scope, affected customers, business criticality, duration, and service-tier policy.

11. Multi-tenancy

Tenant identity is required for ingestion, queries, scenarios, documents, and incident retrieval.

Isolation is repeated at multiple boundaries:

- SQLite queries include tenant predicates.
- Scenario definitions can only run under their declared tenant.
- Qdrant dense and sparse prefetches both include tenant payload filters.
- Redis keys include tenant identity before hashing.
- LangGraph thread IDs are prefixed with tenant identity.
- Incident retrieval requires both tenant and incident ID.
- Tests attempt cross-tenant access and expect failure.

The current API uses `x-tenant-id` for an executable demo. Production should derive tenant and roles from validated JWT claims and use database row-level security as defense in depth.

12. Persistence and checkpointing

SQLite persists documents, chunks, audit events, telemetry events, incidents, reports, and timelines. WAL mode improves local read/write behavior.

LangGraph uses `InMemorySaver` by default. When `AEGIS_RAG_CHECKPOINT_PATH` is configured, it uses `SqliteSaver` with a cross-thread connection. Tests prove that a graph run creates a non-empty checkpoint database.

Production should use PostgreSQL-backed async checkpoints with connection pooling, schema setup during deployment, retention, and encryption.

13. Caching

The local profile uses `NullCache`. The production profile uses Redis with a five-minute TTL.

The cache key includes tenant, top K, and exact question. The response's request ID and workflow metadata are refreshed on a hit. Cache labels do not contain tenant IDs or question text, avoiding high-cardinality and sensitive metrics.

An advanced semantic cache could match similar questions, but it would require conservative similarity thresholds and invalidation tied to document versions.

14. Observability

Every HTTP response receives a request ID. Middleware records method, path, status, and latency using structured JSON logs. Metrics count requests, ingestions, deduplications, queries, abstentions, grounded answers, cache hits, scenarios, incidents, and latency sums/counts.

Liveness answers whether the process is alive. Readiness checks configured storage, Qdrant, and Redis dependencies. Keeping these separate avoids restarting a healthy process merely because a dependency is temporarily unavailable.

OpenTelemetry instrumentation can export spans over OTLP to Jaeger. Production should add explicit spans around retrieval legs, graph nodes, model calls, and persistence.

15. Evaluation

Recall at K

Recall at K asks whether any relevant evidence appears in the first K results. In RAG, missing evidence cannot be repaired by generation.

Mean Reciprocal Rank

For each question, reciprocal rank is `1 / rank_of_first_relevant_result`. MRR averages that value. It rewards placing the first useful result early.

nDCG at K

Normalized Discounted Cumulative Gain rewards relevant results near the top while supporting graded relevance. The current labels are binary.

Current verified results

- Six golden questions.
- Recall at 4: 1.00.
- MRR: 0.9167.
- nDCG at 4: 0.9385.

These are repository benchmark results, not production customer metrics. The dataset is deliberately small and should be expanded before making general quality claims.

16. Test strategy

The suite covers:

- Section-aware chunking and provenance.
- Content-hash idempotency.
- Hybrid retrieval relevance.
- Tenant isolation.
- Prompt-injection filtering.
- Abstention without evidence.
- API contracts and request IDs.

- Retrieval metric calculations.
- LangGraph happy path and rewrite path.
- Durable SQLite checkpoints.
- Qdrant dense-sparse RRF and tenant filtering.
- Incident severity, hypotheses, and approval requirement.
- Failure scenarios, healthy control, expected evidence, and incident persistence.
- End-to-end HTTP health, ingestion, query, scenario, and metrics behavior.

The Qdrant integration test uses embedded local mode. Its payload-index warning is expected because local Qdrant does not use server indexes; tenant filtering behavior is still asserted.

17. Safety and threat model

Prompt injection

Retrieved documents are untrusted. Known instruction-override patterns are excluded. The LLM prompt labels sources as evidence, and citation verification is independent of generation.

Unsafe remediation

The system recommends actions but always returns `approval_required=true`. It does not execute rollback, failover, database mutation, or customer communication.

Cross-tenant leakage

Isolation is enforced at API, store, vector, cache, graph-thread, and scenario boundaries. Production should add signed identity, row-level security, encryption, and policy enforcement.

Sensitive telemetry

Questions and document text are not metric labels. Production should add PII/secret redaction, retention policies, and encrypted raw-object storage.

18. Important trade-offs

Why custom retrieval plus frameworks?

The custom engine proves understanding of ranking and provides a deterministic fallback. LangChain provides interoperability. Qdrant provides scalable vector execution. LangGraph provides explicit orchestration. Each layer has a distinct responsibility.

Why SQLite locally?

It minimizes setup and makes the complete system runnable in CI. It is not presented as a high-write distributed telemetry database.

Why deterministic simulation?

Random incidents are difficult to test. Deterministic scenarios provide causal event order, known alert outcomes, known evidence, and reproducible regression checks.

Why not fully autonomous remediation?

Diagnosis is lower risk than production mutation. Human approval preserves accountability and limits blast radius.

Why not Kafka in the default path?

The repository prioritizes one-command execution. Kafka or Redpanda is a sensible production transport, but requiring it for the local demo would add operational weight without improving the core investigation proof. An event-transport interface is a logical extension.

19. Known limitations

- Tenant identity is a header, not authenticated JWT claims.
- SQLite is not suitable for high-volume telemetry ingestion or multi-worker writes.
- The local embedding provider is deterministic feature hashing, not a semantic foundation embedding model.
- Severity classification is rule-based.
- Alerting supports simple threshold/occurrence rules, not full windowed streaming semantics.
- The golden evaluation set has six questions.
- The local generator creates synthetic events rather than consuming real OpenTelemetry streams.
- Redis, Jaeger, and server Qdrant require Docker and are optional to the zero-service path.
- Remediation is recommendation-only; a human-approval interrupt and execution tools are not implemented.

Owning the project means volunteering these limitations before an interviewer discovers them.

20. Production scaling plan

Phase 1: identity and data durability

- Validate OIDC/JWT tokens and derive tenant/roles from claims.
- Move metadata, audit, incidents, and checkpoints to PostgreSQL.
- Add row-level security and migration tooling.
- Store raw telemetry and source documents in object storage.

Phase 2: event ingestion

- Add Kafka or Redpanda with tenant/service partition keys.
- Normalize OTLP logs, metrics, traces, and deployment webhooks in workers.
- Use idempotent event IDs, consumer groups, retries, and a dead-letter topic.
- Add stream windows and alert state persistence.

Phase 3: retrieval quality

- Use production dense and sparse embedding models.
- Add service/dependency/time filters and temporal scoring.
- Add cross-encoder or late-interaction reranking.
- Expand labeled incidents and perform offline/online evaluation.

Phase 4: safe operations

- Add LangGraph interrupts for approval.
- Use short-lived, scoped tool credentials.
- Sign approval events and write immutable audit records.
- Add canary actions, rollback policies, and post-action verification.

21. How to demo the project

Five-minute demo sequence

1. Show the architecture diagram in README.
2. Run `make test` and point out tenant, graph, Qdrant, checkpoint, and scenario coverage.
3. Run `make eval` and explain each metric honestly.
4. Run `make simulate`.
5. Point to the six generated events and two-sample alert rule.
6. Show the top runbook hypothesis, citations, approval flag, and workflow trace.
7. Run the healthy scenario and show that no incident is created.
8. Open `workflow.py`, `retrieval.py`, and `scenarios.py` for technical discussion.

Two-minute verbal pitch

I built AegisRAG to explore how an AI incident commander could be made testable and safe. The project generates deterministic production-like telemetry for isolated tenants, persists every event, applies alert rules, and opens an incident only when the required samples cross a threshold. A checkpointed LangGraph then performs hybrid retrieval over runbooks and postmortems, grades evidence, retries once if necessary, generates an analysis, and deterministically validates its citations. The local profile is fully offline and reproducible, while the production profile adds Qdrant, Redis, OpenTelemetry, Jaeger, and optional structured OpenAI generation. I deliberately stop at a human approval boundary for rollback or mutation. The retrieval layer is gated by Recall at K, MRR, and nDCG, and the repository includes end-to-end failure and healthy-control tests.

22. Interview questions and strong answers

What did you personally build?

I designed and implemented the full repository independently: domain contracts, ingestion, local and Qdrant retrieval, LangGraph workflow, incident simulator, alert detector, persistence, API, evaluation, tests, Docker profile, observability hooks, and documentation. It is a production-style simulation, not a claim of employer production usage.

Why is this RAG rather than ordinary search?

Retrieval selects tenant-scoped operational evidence, then an answer provider synthesizes an incident analysis constrained to that evidence. The result carries citations and can abstain. Search alone would only return chunks; unconstrained generation would not provide source grounding.

Why use both lexical and dense retrieval?

Incident queries contain exact identifiers such as error codes and deployment versions, where lexical search excels, plus paraphrased symptoms, where dense search helps. RRF combines their rank positions without assuming comparable scores.

Why RRF instead of adding BM25 and cosine scores?

Their raw score distributions are incompatible and vary by query. RRF uses rank, making fusion robust without score calibration. Weighted score fusion would require normalization and tuning on a larger labeled set.

Why apply MMR after RRF?

RRF improves relevance across retrieval legs, while MMR reduces redundancy in the final context. They solve different problems. MMR uses normalized relevance so diversity cannot overwhelm the small RRF score range.

Why LangGraph?

The investigation has explicit state, conditional branching, a bounded retry, checkpointing, and future human interrupts. Those requirements fit a state graph better than a single chain or an unbounded tool-calling loop.

Where is LangChain used?

The custom hybrid engine implements LangChain's BaseRetriever and returns LangChain Document objects. The optional OpenAI provider also uses LangChain's structured-output interface. Core ranking remains visible and testable.

How do you prevent hallucinated citations?

The answer provider emits `[C#]` markers. A later deterministic graph node parses every marker and checks that its index exists in selected evidence. Invalid references force abstention. The model does not grade itself.

How is tenant isolation enforced?

Every storage query includes tenant ID, Qdrant filters both dense and sparse legs, cache keys include tenant, graph thread IDs are tenant-prefixed, scenario execution checks ownership, and incident reads require tenant plus incident ID. Cross-tenant tests verify the boundaries.

Why is `x-tenant-id` not enough for production?

A caller can forge it. Production should derive tenant from a validated JWT issued by an identity provider, check roles/policies, and apply database row-level security for defense in depth.

How are incidents generated?

Validated YAML scenarios declare causal, ordered events and alert expectations. The runner assigns timestamps and IDs, persists each event, and evaluates the configured metric comparator and occurrence count. The second violating sample in the payments scenario opens the incident.

Why require consecutive violating samples?

It reduces transient false positives. A production alert engine would additionally support time windows, missing data, reset behavior, aggregation, and state persistence.

How does the healthy control help?

It proves the system can suppress an incident when normal variance remains below threshold. Testing only failure cases would not measure false-positive behavior.

How is idempotency handled?

Document identity includes tenant, source URI, and SHA-256 content hash, backed by a uniqueness constraint. Repeated delivery returns a deduplication result rather than duplicate chunks.

What happens when evidence is weak?

The graph rewrites the query once using operational vocabulary, retrieves again, and then abstains if evidence remains below threshold. It never loops indefinitely.

How would you make confidence statistically meaningful?

The current confidence is a bounded heuristic derived from retrieval score and is labeled as such. Production confidence should be calibrated against held-out incidents using features such as retrieval relevance, evidence agreement, temporal correlation, and verifier outcomes.

Why does readiness check dependencies but liveness does not?

Liveness determines whether the process should be restarted. Readiness determines whether it should receive traffic. A temporary Redis or Qdrant outage should remove the instance from routing, not cause a restart loop.

What happens if Qdrant succeeds but metadata persistence fails?

The current synchronous demo does not provide a distributed transaction. Production should persist an ingestion job and use an outbox/state machine so indexing is retryable and reconciliation can detect missing or stale vectors.

How would you support millions of events per minute?

Use Kafka or Redpanda for partitioned ingestion, stream processors for windowed aggregation and alert state, object/columnar storage for raw telemetry, PostgreSQL for incidents and audit, Qdrant for operational knowledge, and asynchronous graph workers. The API should not synchronously ingest raw event volume.

How would you invalidate Redis cache entries?

Include a tenant knowledge-version or corpus hash in the key, or publish invalidation events after ingestion. Exact-question TTL caching alone is acceptable for the demo but can serve stale evidence after source changes.

Why not store all telemetry in Qdrant?

Vector stores are appropriate for semantic evidence retrieval, not high-volume metric aggregation or authoritative event history. Telemetry belongs in streaming/time-series/log systems; summaries and operational knowledge belong in retrieval indexes.

How would you evaluate generation?

Add groundedness, citation precision, answer correctness, abstention accuracy, action safety, and root-cause ranking labels. Use deterministic checks where possible and blinded human review or model-graded evaluation only with calibration.

What is the biggest current weakness?

The synthetic dataset is small and the local retrieval/alert models are deliberately simple. The architecture and tests are real, but production claims require larger incident corpora, authenticated identity, streaming ingestion, persistent distributed stores, and load/chaos testing.

What would you build next?

I would first add JWT identity and PostgreSQL row-level security, then a Redpanda event adapter with windowed alert state. After that I would expand the labeled incident corpus and add temporal/dependency-aware retrieval before adding any autonomous action tool.

23. Code-reading path for ownership

Read and explain the files in this order:

1. `domain.py` to understand contracts.
2. `scenarios.py` to understand event-to-incident flow.
3. `ingestion.py` and `store.py` to understand knowledge and persistence.
4. `retrieval.py` to understand ranking mathematics.
5. `langchain_adapter.py` and `qdrant_store.py` to understand provider boundaries.
6. `workflow.py` to understand graph state and routing.
7. `generation.py` and `incidents.py` to understand answer safety and hypothesis output.

8. `api.py` to understand assembly and endpoints.
9. `evaluation.py` and tests to understand proof of behavior.

For every file, answer: what responsibility does it own, what interface does it expose, what failure can occur, and how is that failure tested?

24. Commands to memorize

```
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -e '.[dev]'
cp .env.example .env

make lint
make test
make eval
make simulate
make smoke

python3 -m uvicorn aegis_rag.api:app --reload
docker compose up --build
```

25. Honest resume claims

Safe claims:

- Independently designed and built the project.
- Implemented a production-style multi-tenant simulation.
- Built real LangGraph, Qdrant, Redis, OpenTelemetry, API, evaluation, and test integration.
- Achieved the reported metrics on the included six-question golden set.
- Verified the complete local path with automated and HTTP smoke tests.

Claims to avoid unless you later produce evidence:

- Deployed to a real company production environment.
- Reduced real MTTR by a percentage.
- Served millions of events or users.
- Prevented actual customer incidents.
- Achieved general RAG accuracy based on the small benchmark.

Final ownership checklist

You own this project when you can do all of the following without reading notes:

- Draw the complete data flow.
- Explain why each technology exists.
- Derive the intuition behind BM25, cosine similarity, RRF, MMR, Recall at K, MRR, and nDCG.
- Trace the payments scenario through specific files and state transitions.
- Explain every tenant boundary.
- Explain how citation verification differs from generation.
- State the limitations before being prompted.
- Propose a staged production architecture with clear priorities.

- Run the demo and diagnose a failing quality gate.
- Defend every resume bullet with code and test evidence.